

The Software Engineering Discipline

- **Distinct from other engineering fields**
 - **Components:** Usually needs to be **built from scratch**(從零開始), because each software components has different domain.
 - **Metrics**(指標): Hard to measure the complexity of software.
- **Practitioners & Theoreticians**
 - **Practitioners:** Developing techniques for **immediate application**.
 - **Theoreticians:** Search for underlying principles and theories.
- **Computer Aided Software Engineering(CASE)**
 - **A tool for developing software.**
 1. Project planning
 2. Project management
 3. Documentation
 4. Prototyping and simulation
 5. Interface design
 6. Programming e.g. integrated development environments (IDEs)
- **Professional Organization:** ACM, IEEE, etc.
 - Codes of professional conduct and ethics

The Software Life Cycle

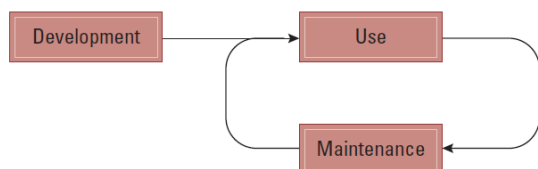
1. Development

1. Analysis
 - Requirements: Application oriented(應用面向)
 - Specifications: Technically oriented
 - Software requirement document
2. Design
 - Plan for the constructions.
3. Implementation
 - Write program, create data files, develop databases
 - Software analyst: requirements and design
 - Programmer: writing codes
4. Testing
 - Quality Assurance(product is compatible with specifications)
 - Debugging

2. Use

3. Maintenance

- debug or update



Software Engineering Methodologies

- **The Waterfall Model**
 - **Development phase is executed strictly.**
 - Hard to deal with changing requirements.
- **Incremental Model**
 - **The first is a simplified version of the final product.**
 - It will be tested and added more functions.
- **Iterative Model**
 - **Improving and refining the existing functions.**
 - Rational Unified Process(RUP) / Unified Process(UP): An iterative model for developing phase.
- **Prototyping**
 - Evolutionary Prototyping
 - The original prototype evolves into the final system.

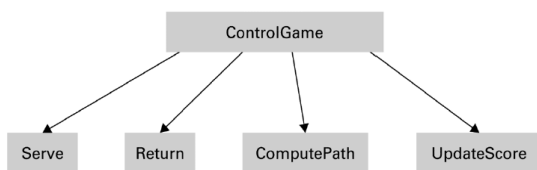
- Throwaway Prototyping:
 - Builds a simplified, imcompleted version to know the product's requirements and design.
 - Be dicarded after know the requirements and design.
- **Open-Source Development**
 1. Authors release the initial version of software on the internet.
 2. Other users can modify or enhance the software.
 3. They report these changes to the original author, who incorporates them into the posted version of the software.
- **Extream Programming(XP)**
 - Follow the **XP Model**.
 - Development stage can be changed anytime.

Modularity(模組化)

- **Introduction**
 - A large software system is divided into many small units.
 - Each unit is called a module. `e.g. class -> modules appear as function

Types of module

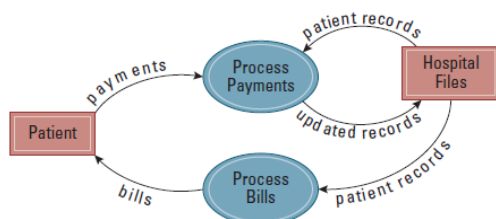
- **Functions:** Imparative paradigm e.g. Function
 - Diagram: Structure charts



- **Objects:** Obeject-oriented e.g. Class
 - Diagram: Collaboration diagrams
 - Provides **single responsibility**.
- **Components:** Components architecture `e.g. A group of classes
 - Diagram: Component Diagrams
 - Provides a **service**.
- **Coupling(耦合性):** The degree of interdependence or connection **between different modules**.
 - **Control coupling:** Occurs when a **module passes control of execution to another**.
 - **Data coupling:** Occurs when a **funciton shares data to another or in the object-oriented design**.
- **Cohesion(內聚性):** The degree of relatedness of a **module's internal parts**.
 - **Logical cohesion:** Some modules have a **similar logic**, but each may can execute **different work**.
 - **Functional cohesion:** All parts of module are focused on a single activity.

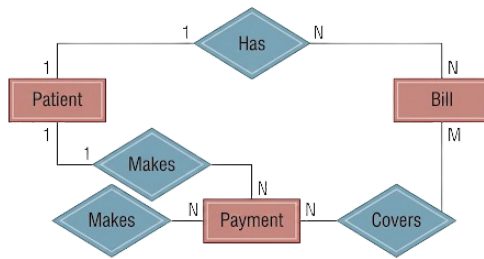
Tools and Trade

- **Dataflow diagram(資料流圖)**
 - Demonstrates how **the data is executed in the system**.
 - Oval: Data manipulation
 - Rectangle: Data source
 - Arrow: path



- **Entity-Relationship diagram**
 - Demonstrates **the relationship between each data**.
 - Type
 - One-to-one relation(1:1)
 - One-to-many relation(1:N)

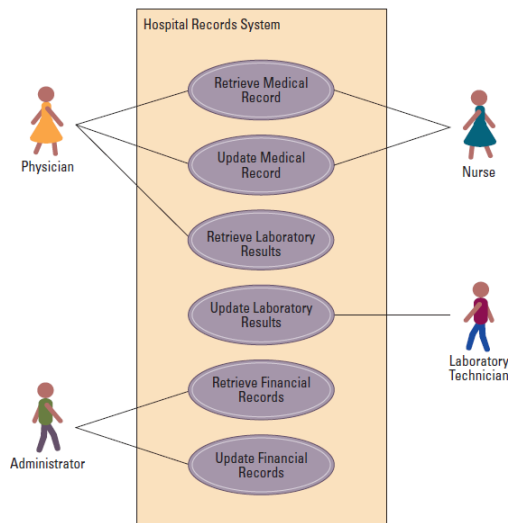
- Many-to-many relation(N:M)



- **Data Dictionary:** A central repository of information about the datas.

Unified Modeling Language(UML)

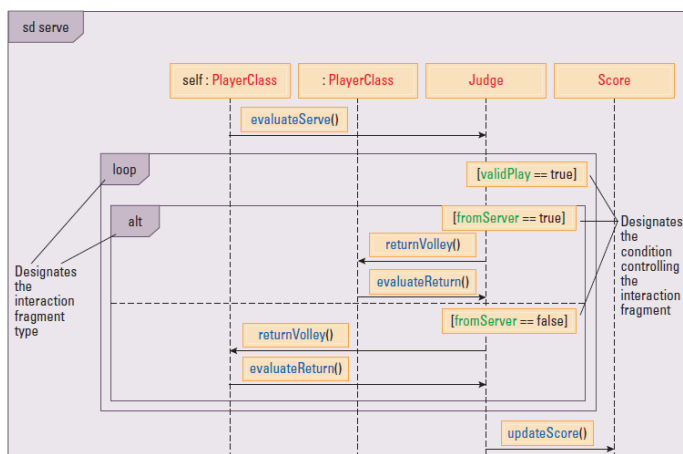
- **Use Case diagram(outside)**
 - **Use cases(Rectangle):** Main system which have some interactions.
 - **Interactions(Oval):** Users can interact with them.
 - **Actors(Stick figures):** Users



- ****Class diagram(inside)**
 - Represents the **structure of classes and relationships** between classes.



- **Sequence diagram:** Expresses dynamic features.



- **CRC(class-responsibility-collaboration) cards:** A physical card is designed to help developers understand object relationships.

Design Patterns

- **A predeveloped model for solving a recurring problem in software design**
- **Adapter pattern:** Used to adapter a module's interface to current needs.
- **Decorator pattern:** Used to control the complexity involved when many different combinations of the same activities are required

Software Testing Strategies

- ****Glass-box testing**
 - All parts of the program needs to be tested.
 - Pareto Principle: Errors usually occurs in some spicific part.
- **Black-box testing**
 - Only focus on input and output.
 - Boudary values analysis
 - **Redundancy**: Different groups developpe same function and test every conclusion are same.
 - Beta testing: Release the software for some users to test.

Documentation

- **User Documentation**: For end user(customers)
- **System Documentation**: For developer
 - Source code comment
 - UML
- **Technical Documentation**: For installing, customizing, updating, etc.

Software Ownership

- **Software Licence**
 - A legal agreement that grants the user certain permissions without transferring ownership.
- **Patents**
 - Must demonstrate that it is new, usable, and not obvious to others with similar background.
 - Process is expensive and time-consuming.